DATA/CS 172: Python for Data Analysis

Functions

Prof. Travis Mandel

9/23/2024

UNIVERSITY of HAWAI'I® HILO

Announcements

- Keep on top of labs
- Program 1 out
  - You have some time to work on it
  - Reminder: Should be your own INDEPENDENT work, as discussed on first week and in syllabus
  - Functions (today’s topic) NOT required for program

Agenda

- Zip wrapup
- Functions

Zip wrapup

- Last time we learned about zip
- Takes two lists, and joins into a list of tuples
- Should mention that it can take *any* number of lists, not just two
- Example
- zip([1,2], [3, 4], [5, 6])
  - → [(1, 3, 5), (2, 4, 6)]

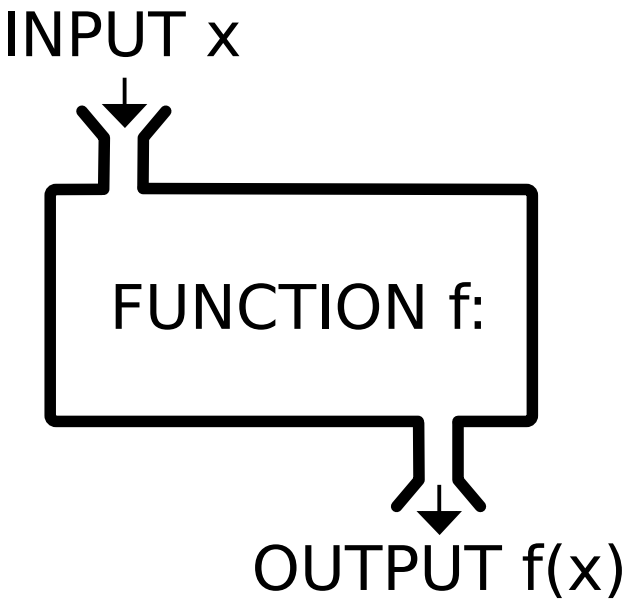
Organization

- So far our scripts have been pretty short…
- For real projects they will get longer!
- Very important to keep stuff organized…
  - Also, there are many things we want to reuse!
- **Functions** give us a way to organize and reuse code
- Separated pieces of code that can be invoked on some parameters

  e.g. len(x) is a function that takes an argument and returns it length

Built-in functions we have seen already

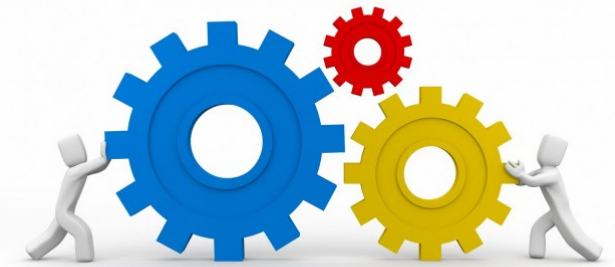
- print()
- open()
- len()
- zip()
- sys.exit()
- There are others…
  - E.g. abs(x) – take the absolute value of x



Components of a function

- **Name**
  - Every function needs a name
  - Name should clearly indicate what a function does (e.g. computeAverage)
- **Arguments**
  - Arguments are variables that come into the function
  - 0 arguments
    - func()
  - 1 argument
    - func(x)
  - 2 arguments
    - func(x,y)
- **Return value**
  - What does the function produce?
  - E.g. math.sqrt(x) returns the square root of x
  - Some functions don’t produce anything
    - E.g. writeInfoToFile()

Defining your own function



Functions start with the keyword def, the body of the function is indented.

def myFunc(arg1, arg2): ← Called the function declaration

    return val ← Called the return statement

Calling the function

ret = myFunc(0,1) ← Called the function call

Note: Argument & return value names do NOT need to be the same in the declaration and the call

    Works on order

Demo: average of two numbers

Returning multiple values

- A pain in most other languages (e.g. C++)
- Easy in Python! (thanks tuples!)
  - return (val1, val2)

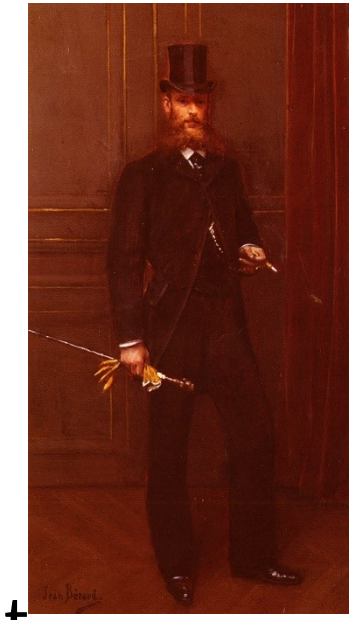
When should I use a function?

- Functions make code more readable and reusable
- "Information hiding" – Don't need to worry about details of how function works when I am trying to understand the general flow
- Should use a function if:
  - A piece of code serves a coherent purpose
  - A piece of code is fairly complex (simply multiplying two numbers doesn't merit a function)
- Especially useful when code needs to be repeated in several places

Things you should NOT do with functions

- Python (usually) lets functions use whatever variables they want
  - NOT good – functions should only use their arguments, or any new variables you make inside of them
  - Messy + things can go wrong here
- Python lets you put whatever code you want above functions
  - NOT good – functions should be at the TOP of your script
    - Generally, the only thing above them should be import statements

More Style notes with functions



- Should have a descriptive name that indicates what it does
  - computeTotalPrice() instead of func1()
- Arguments should be named well to indicate what they represent
- Usually bad to have **printing** happen inside the function
  - Except for helping track down problems etc.
  - Printing should happen in the main part of your code
  - If you need something to print from your function, it should be returned

Lab out

- Easing you into functions
  - Modify previous lab to make use of a function
  - Will see a bit more complex example next time